

4. Neither the primal nor the dual is feasible.

29.3-8

Prove the fundamental theorem of linear programming, Theorem 29.5.

Problems

29-1 *Linear-inequality feasibility*

Given a set of m linear inequalities on n variables $x_1, x_2, \dots, x_n$, the *linear-inequality feasibility problem* asks whether there is a setting of the variables that simultaneously satisfies each of the inequalities.

- a. Given an algorithm for the linear-programming problem, show how to use it to solve a linear-inequality feasibility problem. The number of variables and constraints that you use in the linear-programming problem should be polynomial in n and m .
- b. Given an algorithm for the linear-inequality feasibility problem, show how to use it to solve a linear-programming problem. The number of variables and linear inequalities that you use in the linear-inequality feasibility problem should be polynomial in n and m , the number of variables and constraints in the linear program.

29-2 *Complementary slackness*

Complementary slackness describes a relationship between the values of primal variables and dual constraints and between the values of dual variables and primal constraints. Let $\bar{x}$ be a feasible solution to the primal linear program given in (29.31)–(29.33), and let $\bar{y}$ be a feasible solution to the dual linear program given in (29.34)–(29.36). Complementary slackness states that the following conditions are necessary and sufficient for $\bar{x}$ and $\bar{y}$ to be optimal:

$$\sum_{i=1}^m a_{ij} \bar{y}_i = c_j \text{ or } \bar{x}_j = 0 \text{ for } j = 1, 2, \dots, n$$

and

$$\sum_{j=1}^n a_{ij} \bar{x}_j = b_i \text{ or } \bar{y}_i = 0 \text{ for } i = 1, 2, \dots, m.$$

- a.** Verify that complementary slackness holds for the linear program in lines (29.37)–(29.41).
- b.** Prove that complementary slackness holds for any primal linear program and its corresponding dual.
- c.** Prove that a feasible solution $\bar{x}$ to a primal linear program given in lines (29.31)–(29.33) is optimal if and only if there exist values $\bar{y} = (\bar{y}_1, \bar{y}_2, \dots, \bar{y}_m)$ such that
 1. $\bar{y}$ is a feasible solution to the dual linear program given in (29.34)–(29.36),
 2. $\sum_{i=1}^m a_{ij} \bar{y}_i = c_j$ for all j such that $\bar{x}_j > 0$, and
 3. $\bar{y}_i = 0$ for all i such that $\sum_{j=1}^n a_{ij} \bar{x}_j < b_i$.

29-3 Integer linear programming

An *integer linear-programming problem* is a linear-programming problem with the additional constraint that the variables x must take on integer values. Exercise 34.5-3 on page 1098 shows that just determining whether an integer linear program has a feasible solution is NP-hard, which means that there is no known polynomial-time algorithm for this problem.

- a.** Show that weak duality (Lemma 29.1) holds for an integer linear program.
- b.** Show that duality (Theorem 29.4) does not always hold for an integer linear program.
- c.** Given a primal linear program in standard form, let P be the optimal objective value for the primal linear program, D be the optimal objective value for its dual, IP be the optimal objective value for the integer version of the primal (that is, the primal with the added constraint that the variables take on integer values), and ID be the

optimal objective value for the integer version of the dual. Assuming that both the primal integer program and the dual integer program are feasible and bounded, show that

$$IP \leq P = D \leq ID.$$

29-4 *Farkas's lemma*

Prove Farkas's lemma, Lemma 29.3.

29-5 *Minimum-cost circulation*

This problem considers a variant of the minimum-cost-flow problem from Section 29.2 in which there is no demand, source, or sink. Instead, the input, as before, contains a flow network, capacity constraints $c(u, v)$, and edge costs $a(u, v)$. A flow is feasible if it satisfies the capacity constraint on every edge and flow conservation at *every* vertex. The goal is to find, among all feasible flows, the one of minimum cost. We call this problem the *minimum-cost-circulation problem*.

- a. Formulate the minimum-cost-circulation problem as a linear program.
- b. Suppose that for all edges $(u, v) \in E$, we have $a(u, v) > 0$. What does an optimal solution to the minimum-cost-circulation problem look like?
- c. Formulate the maximum-flow problem as a minimum-cost-circulation problem linear program. That is, given a maximum-flow problem instance $G = (V, E)$ with source s , sink t and edge capacities c , create a minimum-cost-circulation problem by giving a (possibly different) network $G' = (V', E')$ with edge capacities c' and edge costs a' such that you can derive a solution to the maximum-flow problem from a solution to the minimum-cost-circulation problem.
- d. Formulate the single-source shortest-path problem as a minimum-cost-circulation problem linear program.

This chapter only begins to study the wide field of linear programming. A number of books are devoted exclusively to linear programming, including those by Chvátal [94], Gass [178], Karloff [246], Schrijver [398], and Vanderbei [444]. Many other books give a good coverage of linear programming, including those by Papadimitriou and Steiglitz [353] and Ahuja, Magnanti, and Orlin [7]. The coverage in this chapter draws on the approach taken by Chvátal.

The simplex algorithm for linear programming was invented by G. Dantzig in 1947. Shortly after, researchers discovered how to formulate a number of problems in a variety of fields as linear programs and solve them with the simplex algorithm. As a result, applications of linear programming flourished, along with several algorithms. Variants of the simplex algorithm remain the most popular methods for solving linear-programming problems. This history appears in a number of places, including the notes in [94] and [246].

The ellipsoid algorithm was the first polynomial-time algorithm for linear programming and is due to L. G. Khachian in 1979. It was based on earlier work by N. Z. Shor, D. B. Judin, and A. S. Nemirovskii. Grötschel, Lovász, and Schrijver [201] describe how to use the ellipsoid algorithm to solve a variety of problems in combinatorial optimization. To date, the ellipsoid algorithm does not appear to be competitive with the simplex algorithm in practice.

Karmarkar's paper [247] includes a description of the first interior-point algorithm. Many subsequent researchers designed interior-point algorithms. Good surveys appear in the article of Goldfarb and Todd [189] and the book by Ye [463].

Analysis of the simplex algorithm remains an active area of research. V. Klee and G. J. Minty constructed an example on which the simplex algorithm runs through $2^n - 1$ iterations. The simplex algorithm usually performs well in practice, and many researchers have tried to give theoretical justification for this empirical observation. A line of research begun by K. H. Borgwardt, and carried on by many others, shows that under certain probabilistic assumptions on the input, the simplex algorithm converges in expected polynomial time. Spielman and Teng

[421] made progress in this area, introducing the “smoothed analysis of algorithms” and applying it to the simplex algorithm.

The simplex algorithm is known to run efficiently in certain special cases. Particularly noteworthy is the network-simplex algorithm, which is the simplex algorithm, specialized to network-flow problems. For certain network problems, including the shortest-paths, maximum-flow, and minimum-cost-flow problems, variants of the network-simplex algorithm run in polynomial time. See, for example, the article by Orlin [349] and the citations therein.

¹ An intuitive definition of a convex region is that it fulfills the requirement that for any two points in the region, all points on a line segment between them are also in the region.